

GitHub Repository: https://github.com/yosammy01/CSC8830_COMPUTER_VISION/tree/main/MODULE2

Web Application: https://yosammy01.github.io/CSC8830_COMPUTER_VISION/#/module2

- I. Step 1: Perform the camera calibration using built-in tools in OpenCV or MATLAB – use a smartphone camera for this assignment.
 - a. I created and printed out a calibration checkerboard image from an online tool. The printout was taped to the wall.
 - b. I took pictures of the calibration images with my smart phone camera, making sure to maintain a similar distance between the camera and wall, and changed the orientation of the camera to cover as many angles as possible. A total of 14 calibration images were obtained.
 - c. Checkerboard calibration images were placed in the 'CSC8830_COMPUTER_VISION\MODULE2\images' directory.
 - d. I executed the script from the terminal: "python calibration_code.py"
 - e. The script generated 'calibration_params.json' with the camera-matrix values, which is required by the dimension measurement script in Step 2.
 - i. camera_matrix = calibration matrix = Intrinsic Matrix.
 - ii. Distortion coefficients were also produced to account for distortion in the camera lenses.
- II. Step 2: Implement in Python or MATLAB, a script to find the real world 2D dimensions of an object using perspective projection equations.
 - a. Using the known computer vision formulas and the f_x and f_y from the Intrinsic matrix, the real-world dimensions of the objects could be found:
$$X_{world} = (X_{im} * Z_{world}) / f_x$$
$$Y_{world} = (Y_{im} * Z_{world}) / f_y$$
 - b. Various filtering techniques were used since many objects had trouble being detected.
- III. Step 3: Validate using an experiment where you image an object using your camera from a specific distance (choose any distance greater than 2 meters and ensure you are able to measure it accurately) between the object and camera. Validation must be done for 20 different object measurements. Report your error estimates statistics.
 - a. Ensured 'calibration_code.py' ran first so that 'calibration_params.json' exists.
 - b. Placed 20 measurement images in the 'CSC8830_COMPUTER_VISION\MODULE2\measurements' directory. I used the same distance from camera to object (2133.6 mm) for all measurement images. This is the same Z_{world} value used in the script.
 - c. Executed the script from the terminal: "python get_real_world_2d_dimensions_code.py"
 - d. The script displayed bounding boxes over the detected objects. I pressed any key to proceed to the next image.
 - e. A 'dimensions_output.txt' was produced by the script, with the real-world dimensions of each object detected in the 20 measurement images.

Image Name	Real World Width (mm)	Real World Height (mm)
air_purifier.jpg	282.6	381
black_box.jpg	230.2	277.8

book.jpg	112.7	177.8
box.jpg	254	336.5
brick.jpg	203.2	92.07
catalog_case.jpg	457.2	379.41
clock_radio.jpg	107.9	127
large_circle_lid.jpg	190.5	190.5
large_rectangle_lid.jpg	247.6	225.4
remote_control.jpg	169.86	49.21
rice_cooker.jpg	214.31	157.16
sandal.jpg	295.27	95.25
shoe_box.jpg	330.2	114.3
small_circle_lid.jpg	152.4	152.4
small_rectangle_lid.jpg	206.4	133.3
small_safe.jpg	323.85	254
smart_phone.jpg	79.38	158.75
speaker_box.jpg	206.38	214.31
sponge.jpg	104.77	61.91
table_top.jpg	481	368.3

Image Name	Projected Width (mm)	Projected Height (mm)
air_purifier.jpg	261.99	356.42
black_box.jpg	222.9	271.95
book.jpg	109.05	175.12
box.jpg	242.79	331.01
brick.jpg	194.78	92.71
catalog_case.jpg	437.56	359.85
clock_radio.jpg	103.56	124.99
large_circle_lid.jpg	338.12	343.37
large_rectangle_lid.jpg	236.61	190.91
remote_control.jpg	161.86	49.45
rice_cooker.jpg	196.15	149.02
sandal.jpg	281.19	93.4
shoe_box.jpg	316.17	117.43
small_circle_lid.jpg	148.14	152.46
small_rectangle_lid.jpg	185.18	142.16
small_safe.jpg	306.57	249.97
smart_phone.jpg	76.13	155.2
speaker_box.jpg	200.95	208.77
sponge.jpg	100.13	62.49
table_top.jpg	458.14	348.18

Image Name	Percent Error (Width)	Percent Error (Height)
air_purifier.jpg	7.29	6.45
black_box.jpg	3.17	2.11
book.jpg	3.24	1.51
box.jpg	4.41	1.63
brick.jpg	4.14	0.7
catalog_case.jpg	4.3	5.16
clock_radio.jpg	4.02	1.58
large_circle_lid.jpg	77.49	80.25
large_rectangle_lid.jpg	4.44	15.3
remote_control.jpg	4.71	0.49
rice_cooker.jpg	8.47	5.18
sandal.jpg	4.77	1.94
shoe_box.jpg	4.25	2.74
small_circle_lid.jpg	2.8	0.04
small_rectangle_lid.jpg	10.28	6.65
small_safe.jpg	5.34	1.59
smart_phone.jpg	4.09	2.24
speaker_box.jpg	2.63	2.59
sponge.jpg	4.43	0.94
table_top.jpg	4.75	5.46

Statistic	Percent Error (Width)	Percent Error (Height)
Count	20	20
Mean (Average)	8.45%	7.23%
Median	4.42%	2.18%
Standard Deviation	16.36%	17.53%
Minimum Error	2.63%	0.04%
Maximum Error	77.49%	80.25%

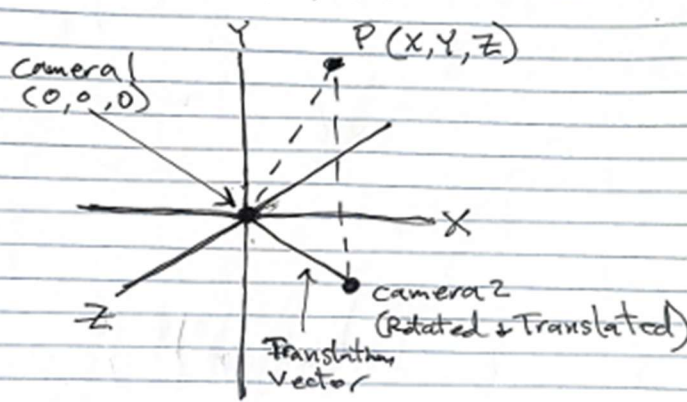
NOTE: Removing the result for the object with the highest percent error.		
Statistic	Percent Error (Width)	Percent Error (Height)
Count	19	19
Mean (Average)	4.82%	3.38%
Median	4.41%	2.11%
Standard Deviation	1.92%	3.53%
Minimum Error	2.63%	0.04%
Maximum Error	10.28%	15.30%

- IV. Theory: Derive the relationship (mathematically) between the image coordinates of a Point $P(X,Y,Z)$ in space on camera 1 and camera 2 where camera 1 is static and camera 2 is situated at a certain distance and at an oblique orientation from camera 1. Point P is within the FoV of both the cameras. Make assumptions as necessary and justify the same. Provide clarity on all static parameters, variables and how they can be computed and/or determined.

Samuel
Robles
CSC 8830
Module 2

Theory:

① Assumptions include: (1) Pinhole Camera Model, (2) Undistorted Images, and (3) Static World Frame - camera 1 represents the origin of the 3D world $(0,0,0)$.



② P_i is point P with respect to camera 1 (camera 1's perspective).

$$P_i = \begin{bmatrix} X_i \\ Y_i \\ Z_i \end{bmatrix}$$

- ③ Camera2 is Camera1's perspective that is rotated and translated, P_2 is point P with respect to camera2 (camera2's perspective).

$$P_2 = RP_1 + t$$

↑ ↑
Rotation translation
Matrix vector

- ④ Camera1, Camera2, and P form an "Epipolar Plane". The line connecting camera1 and camera2 is the translation vector. Using "Epipolar Geometry" we can use the "Epipolar Constraint" to derive the relationship between image coordinates of P on camera1 and camera2.

a. The cross-product of the translation vector with itself, equals 0.

$$t \times P_2 = t \times RP_1 + t \times t$$

$$t \times P_2 = t \times RP_1$$

b. We know $a \times B = [a]_{\times} B$
So, $t \times P_2 = [t]_{\times} P_2$, we convert
 t into a "skew-symmetric" matrix.

$$t = \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix} \Rightarrow [t]_{\times} = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}$$

$$[t]_{\times} P_2 = [t]_{\times} R P_1$$

c. Since $t \times P_2$ gives a normal
vector perpendicular to t and
 P_2 , so does $[t]_{\times} P_2$.

$$P_2^T \cdot ([t]_{\times} P_2) = 0.$$

($a \cdot b = \|a\| \|b\| \cos(\theta)$,
and the angle between P_2^T
and $([t]_{\times} P_2)$ is 90° .)

$$P_2^T \cdot ([t]_{\times} P_2) = P_2^T [t]_{\times} R P_1$$

$$P_2^T [t]_{\times} R P_1 = 0$$

d. $\begin{bmatrix} t \\ R \end{bmatrix}_x$ combines the rotation and translation into a single "Essential Matrix" E .

$$\begin{bmatrix} t \\ R \end{bmatrix}_x R = E$$

$$P_2^T E P_1 = 0 \quad (\text{Epipolar Constraint})$$

e. We know for each camera

$$P_1 = Z_1 K_1^{-1} p_1$$

$$P_2 = Z_2 K_2^{-1} p_2$$

(p_1 and p_2 are the image vectors) for camera 1 and camera 2.

$$(Z_2 K_2^{-1} p_2)^T E (Z_1 K_1^{-1} p_1) = 0$$

$$Z_2 p_2^T K_2^{-T} E Z_1 K_1^{-1} p_1 = 0$$

$$Z_2 Z_1^T (p_2^T K_2^{-T} E K_1^{-1} p_1) = 0$$

$$p_2^T K_2^{-T} E K_1^{-1} p_1 = 0$$

f. $K_2^{-T} E K_1^{-1}$ is the "Fundamental Matrix" F .

$$p_2^T F p_1 = 0$$